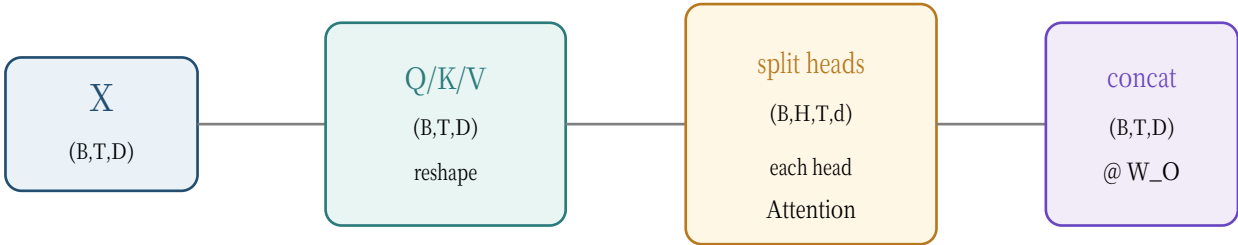


Mask 与 Multi-Head Attention

为什么一个 Attention 要拆成多个 Head

本讲定位 第 54 讲已经完成单个 Scaled Dot-Product Attention。本讲只做两件事：第一，把 Mask 的 Shape 与广播规则真正讲清楚；第二，把一个 D_{model} 维 Attention 拆成多个 Head，再 concat 并通过 Output Projection 合回来。

Multi-Head Attention 数据流



Head 是拆 feature dimension，不是拆 Token；所有 Head 都看到同一序列位置。

必须真正掌握	验收标准
Head splitting	能从 (B, T, D) 推到 $(B, H, T, d_{\text{head}})$
Mask broadcasting	知道 causal / padding mask 怎样扩展到 B/H
Per-head Attention	知道每个 Head 有独立 score / weight / output
Concat + W_O	能恢复 (B, T, D) 并解释 W_O 的作用
参数量	知道 D 固定时增加 Head 数不会增加标准 QKV/O 参数量

1. 为什么要 Multi-Head，而不是一个大 Head

单个 Attention 可以让每个 Token 动态读取其他 Token，但所有匹配都发生在同一个 Query/Key 子空间里。Multi-Head 的基本想法是：把总特征维 D_{model} 分成 H 个较小的 head dimension，让多组 Attention 并行计算。

$$D_{\text{model}} = H * d_{\text{head}}$$

例如：

$$D_{\text{model}}=768, H=12, d_{\text{head}}=64$$

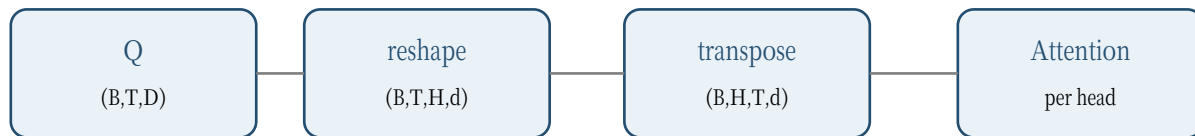
每个 Head 都拥有自己的 Q/K/V 子空间，并产生自己的 $T \times T$ Attention Pattern。最终再把多个 Head 的输出拼回 D_{model} 。

不要过度语义化

可以把多个 Head 理解成“允许模型同时使用多种匹配子空间”，但不能预先断言 Head1 一定学语法、Head2 一定学实体。真实模型中 Head 可能专门化，也可能高度冗余。

2. Head 拆的不是 Token，而是 Feature Dimension

$$D_{\text{model}} = H \times d_{\text{head}}$$



reshape 改“如何分组”，transpose 改“轴的顺序”

二者都不会凭空产生新数值，但语义完全不同

这是初学者最容易搞错的一点：

$(B, T, D_{\text{model}}) \rightarrow (B, T, H, d_{\text{head}}) \rightarrow (B, H, T, d_{\text{head}})$

Token 数 T 没有被分给不同 Head。每个 Head 都仍然看到全部 T 个 Token，只是每个 Head 使用自己那一份 projected feature dimensions。

真实模型更精确
的说法

标准实现通常先用一个完整 $D_{\text{model}} \rightarrow D_{\text{model}}$ Projection；每个 Head 是投影结果中的一个 slice。由于 Projection Matrix 可以混合所有输入特征，所以 Head 并不只是固定读取原始输入的某几个维度。

3. reshape 和 transpose 分别做了什么

假设：

$$Q.shape = (B,T,D) = (1,3,4), H=2, d_head=2$$

第一步 reshape：

$$(1,3,4) \rightarrow (1,3,2,2) = (B,T,H,d_head)$$

只是把最后一个 $D=4$ 重新解释成 $H \times d_head = 2 \times 2$ 。然后 transpose：

$$(B,T,H,d) \rightarrow (B,H,T,d)$$

这样做的目的，是把 Head 轴放到 Sequence 轴前面，便于每个 Head 独立做：

$$(T,d) @ (d,T) \rightarrow (T,T)$$

```
Q = Q.reshape(B, T, H, d_head)
```

```
Q = Q.transpose(1, 2) # (B,H,T,d_head)
```

工程细节

transpose 后 Tensor 常变成 non-contiguous。后面若要 view 回去，常需要 contiguous()；reshape 在很多情况下会自动处理。理解语义比机械背 API 更重要。

4. 一个极小的 2-Head 数字例子

实验用：

X.shape=(1,3,4), H=2, d_head=2	
Token	4维表示
1	[1,0,0,1]
2	[0,1,0,1]
3	[1,1,1,0]

为了把 Head splitting 看清楚，本实验先令 $Q=K=V=X$ ，相当于 identity projection。于是：

Head	Token1	Token2	Token3
Head1 前两维	[1,0]	[0,1]	[1,1]
Head2 后两维	[0,1]	[0,1]	[1,0]

为什么只是教学例子

真实模型不会要求 Head1 永远只看原始前两维；可学习 Projection 会先重新混合特征，再切成多个 Head。

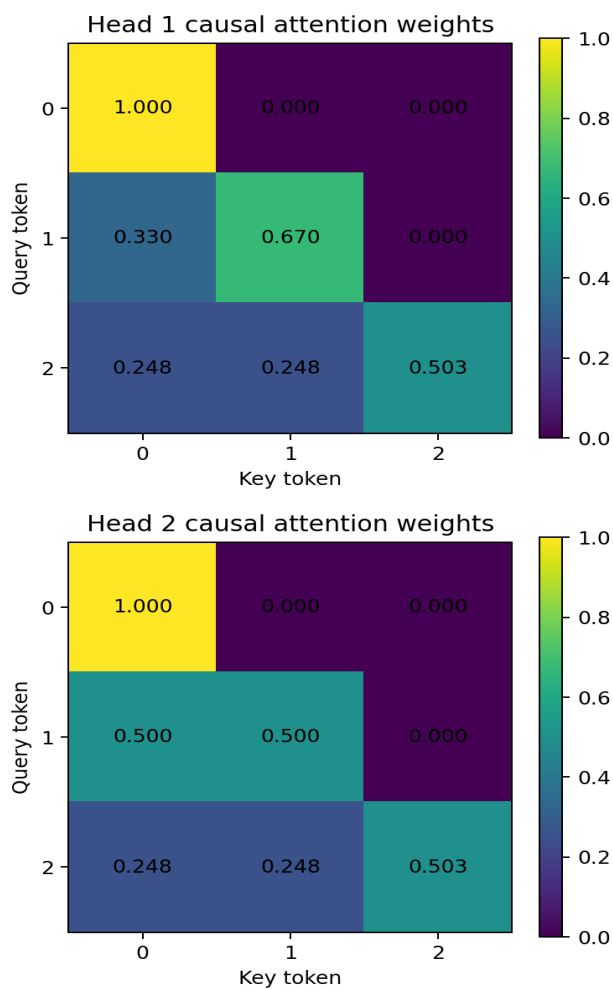
5. 每个 Head 独立执行完整 Attention

$$\text{Scores}_h = Q_h K_h^T / \sqrt{d_{\text{head}}}$$

$$\text{Weights}_h = \text{Softmax}(\text{MaskedScores}_h)$$

$$O_h = \text{Weights}_h V_h$$

因为两个 Head 的 Q/K 数值不同，所以它们即使面对完全相同的 Token 位置，也可以产生不同的 Attention Pattern。



Multi-Head
的核心

多个 Head 不是对同一个 Attention 结果做复制；每个 Head 有自己的 Q/K/V slice，因此独立产生 score、softmax weight 和 output。

6. 为什么 Scaling 用 $\sqrt{d_{\text{head}}}$, 不是 $\sqrt{D_{\text{model}}}$

第 54 讲的 scaling 来自 Dot Product 的维度。每个 Head 的 score 是：

$$q_h(d_{\text{head}}) \cdot k_h(d_{\text{head}})$$

所以方差增长取决于 d_{head} , 而不是完整 D_{model} 。标准 Multi-Head 中：

$$\text{scale} = 1 / \sqrt{d_{\text{head}}}$$

例如 $D_{\text{model}}=768$ 、 $H=12$ ：

$$d_{\text{head}}=64, \text{scale}=1/8$$

常见错误

如果已经拆成 12 个 64 维 Head, 却仍除以 $\sqrt{768}$, 会把 score 过度缩小, 改变 Softmax 分布。

7. Mask 怎样扩展到多个 Head

Mask Broadcasting：同一规则如何应用到 Batch 与 Head



broadcast 的目的：不复制逻辑含义，只把同一可见性规则扩展到需要的维度

标准 causal mask 的核心只依赖 Query/Key 位置，所以逻辑上只需要：

(T,T)

它可以广播到：

(B,H,T,T)

Padding Mask 则通常依赖样本，因为每条序列 PAD 长度可能不同。常见逻辑 Shape 可以写成：

(B,1,1,T)

再与 causal mask 组合，得到每个样本/Head/Query 的最终可见 Key 集合。

框架语义警告

不同 PyTorch API 的 bool mask 语义并不完全统一。科研代码里不要只看 True/False，要查 API 文档或写最小数值测试确认 True 到底代表“允许”还是“屏蔽”。

8. Causal Mask + Padding Mask 的数值实验

实验构造：

```
scores.shape=(B,H,T,T)=(2,2,4,4)
```

Causal Mask 是 (4,4)；Padding Mask 是 (2,1,1,4)，第二条样本最后一个 Key 是 PAD。通过广播组合后：

```
combined = causal OR padding
```

Softmax 后第二条样本所有 Head、所有 Query 对最后 PAD Key 的权重最大值为 0。

这说明：Mask 并不需要显式复制成巨大 $B \times H \times T \times T$ 数组才能表达规则；广播可以复用逻辑结构。

LLM 连接

训练批次中如果不同样本存在 PAD，通常既需要因果约束，又要避免真实 Token 关注 PAD。具体实现取决于 tokenizer、packing 和框架。

9. 多个 Head 算完以后为什么还要 concat

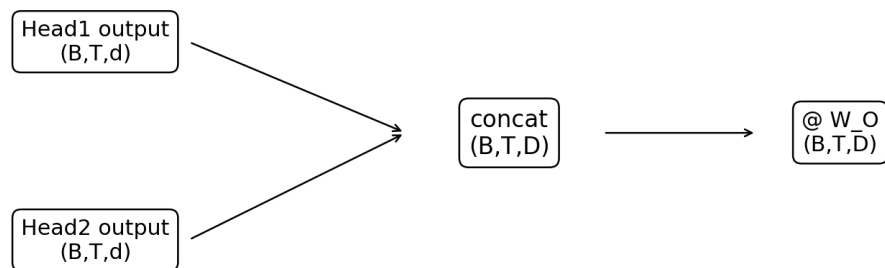
每个 Head 输出：

$$O_h.shape=(B,T,d_{head})$$

H 个 Head 拼起来：

$$\text{Concat}(O_1,\dots,O_H).shape=(B,T,H*d_{head})=(B,T,D_{model})$$

Heads are recombined only after independent attention



这样恢复到 D_{model} ，才能继续走残差连接和后续 Transformer 子层。

concat 本身没有
学习参数

concat 只是把多个 Head 的 feature slices 摆回同一个向量。真正负责“重新混合各 Head 信息”的是后面的 Output Projection W_O 。

10. Output Projection W_O 到底做什么

$$O = \text{Concat}(\text{head}_1, \dots, \text{head}_H) @ W_O$$

其中常见：

$$W_O.\text{shape} = (D_{\text{model}}, D_{\text{model}})$$

它有两个作用：第一，把多个 Head 的拼接结果重新线性混合；第二，让输出仍保持 D_{model} ，便于与 Residual Stream 相加。

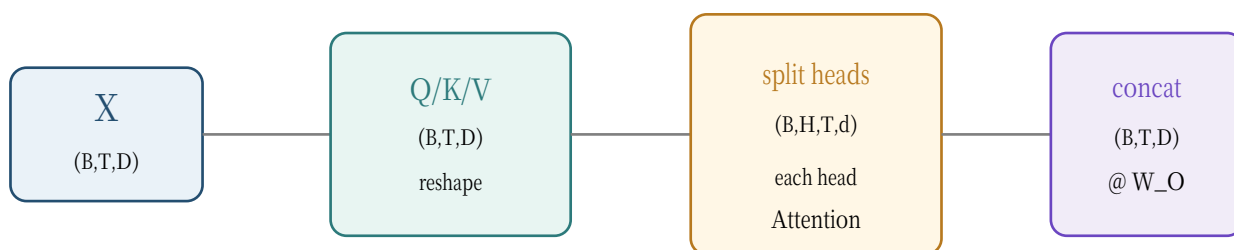
实验中我们固定一个非单位 W_O ，并验证最终输出与 PyTorch `nn.MultiheadAttention` 完全对齐。

如果 W_O 是单位矩阵会怎样 多个 Head 仍然完成独立 Attention，只是 concat 后不再做额外线性重混合。标准 Transformer 通常保留可训练 W_O 。

11. 整个 Multi-Head Attention 的 Shape 主线

X: (B,T,D)
Q,K,V: (B,T,D)
reshape -> (B,T,H,d)
transpose -> (B,H,T,d)
Scores/Weights -> (B,H,T,T)
Head outputs -> (B,H,T,d)
transpose -> (B,T,H,d)
reshape/concat -> (B,T,D)
@ W_O -> (B,T,D)

Multi-Head Attention 数据流



Head 是拆 feature dimension，不是拆 Token；所有 Head 都看到同一序列位置。

最重要的 Shape 检查

D_{model} 必须能被 num_heads 整除，才能得到整数 d_{head} 。标准 `nn.MultiheadAttention` 会直接要求 `embed_dim % num_heads == 0`。

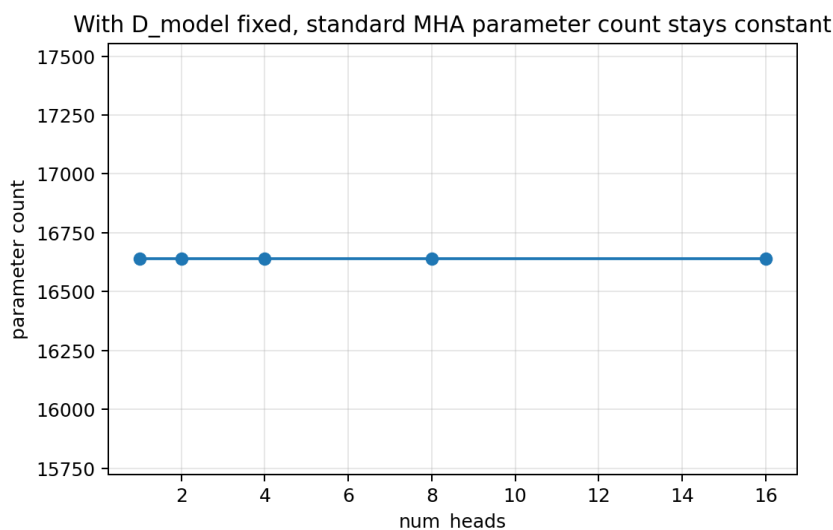
12. 为什么增加 Head 数通常不增加标准 MHA 参数量

在 D_{model} 固定时，标准 Q/K/V/O Weight 仍然是：

W_Q, W_K, W_V, W_O each roughly $D \times D$

total weight params $\approx 4D^2$

把 D 切成 1 个 Head 还是 12 个 Head，主要改变 d_{head} ，不改变这四个大矩阵的总尺寸。



PyTorch 含 bias 的标准 MultiheadAttention 参数量为：

$$4D^2 + 4D$$

$D=768$ 时：

2,362,368 parameters

但运行时结构会变

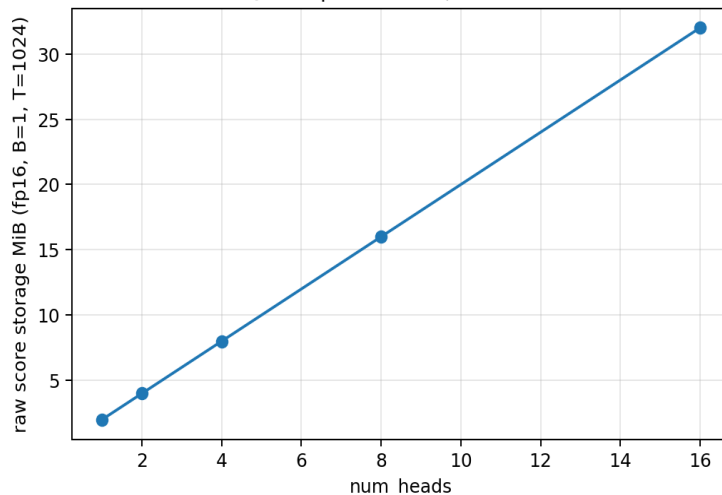
参数量近似不变，不代表显存/数值行为都不变。Head 数改变会改变 d_{head} ，并产生更多独立的 $T \times T$ score/weight maps。

13. 更多 Head 也会付出代价

标准 score 计算总 FLOPs 在 D 固定时大致仍为 $O(T^2D)$ ，但 score tensor 数量是 H 个：

`Scores.shape=(B,H,T,T)`

More heads do not add QKV/O parameters, but create more T×T score maps



例如只看 raw fp16 score storage、B=1、T=1024：Head 数越多，需要显式存放的 $T \times T$ map 越多。现代 fused attention 会减少中间 materialization，但逻辑 Shape 仍然存在。

更多 Head
不等于更强

H 增大意味着 d_{head} 变小；过多 Head 可能带来冗余、过小子空间或额外实现成本。Head 数是架构超参数，不是单调越大越好。

14. PyTorch nn.MultiheadAttention 里面到底有什么

典型调用：

```
mha = nn.MultiheadAttention(  
    embed_dim=D,  
    num_heads=H,  
    batch_first=True  
)  
  
out, weights = mha(  
    x, x, x,  
    attn_mask=mask,  
    need_weights=True,  
    average_attn_weights=False  
)
```

PyTorch 常把 Q/K/V Projection 参数融合保存为：

```
in_proj_weight.shape = (3D, D)
```

然后还有：

```
out_proj.weight.shape = (D,D)
```

实验把 in_proj_weight 设成三份 Identity，并固定 out_proj，使 PyTorch 与手写 Multi-Head causal attention 最大误差仅为浮点误差。

**need_weights
的科研代价**

如果只是训练/推理，不一定需要显式返回 Attention Weights。请求权重可能影响某些高效实现路径；做分析时再打开，并明确平均 Head 还是保留每个 Head。

15. Self-Attention、Cross-Attention 与 Multi-Head 是不同维度

概念	Q 来自哪里	K/V 来自哪里	是否可以 Multi-Head
Self-Attention	同一序列 X	同一序列 X	可以
Cross-Attention	序列 A / decoder state	另一个序列/encoder memory	可以
Multi-Head	一种并行子空间组织方式	不决定数据来源	本身就是 Head 结构

因此“Multi-Head Self-Attention”同时说明了两件事：Q/K/V 来自同一序列，并且 Attention 被拆成多个 Head。

Agent Systems 连接

在一些多模态 Agent 或 encoder-decoder 系统里，你会看到 Cross-Attention：Query 可能来自当前语言状态，K/V 来自视觉特征、检索结果或其他 memory。底层仍是同样的 $QK^T \rightarrow \text{Softmax} \rightarrow V$ 。

16. Head 是否真的会学到不同 “ 功能 ”

Multi-Head 给模型提供了多个独立的投影与 Attention Pattern，因此 “ 有能力 ” 学习不同关系。但这不等于每个 Head 必然形成清晰可命名功能。真实研究中经常发现：

现象	含义
部分 Head 很有结构	可能对某类依赖/位置关系敏感
部分 Head 高度相似	存在冗余
删除某些 Head 影响不大	并非所有 Head 同等重要
Attention pattern 随层变化	不能把单层单 Head 结论外推到整模型
科研判断	如果论文声称 “ 某 Head 负责某功能 ”，最好还要看 ablation、intervention、跨样本稳定性，而不是只展示一张漂亮 Attention Heatmap。

17. Mask 的三类常见研究错误

错误 1：把 bool mask 的 True/False 语义想当然。不同 API 可能相反。

错误 2：Padding 只在 Loss 里忽略，却让 Attention 仍读取 PAD。

这取决于模型/任务设计，不能机械认为二者等价。

错误 3：把 causal mask 与 sequence boundary 搞混。packing 多条序列时，如果边界 mask 处理错误，前一条样本可能泄漏到后一条。

对 Agent trajectory
特别重要

如果把多个 trajectory 打包到同一个长序列里，必须明确哪些位置允许互相 Attention。错误的 boundary mask 会造成训练信息泄漏，得到虚假的效果。

因此做 LLM / Agent 实验时，Mask 不只是“代码细节”，而是实验协议的一部分。

18. GQA / MQA 预览：不是所有模型都让 Q/K/V Head 数相同

标准 MHA 常见：

```
num_query_heads = num_key_heads = num_value_heads
```

现代 LLM 为降低 KV Cache 和推理成本，经常使用：

结构	Q heads	K/V heads	直觉
MHA	多	同样多	每个 Q head 有独立 K/V head
GQA	多	更少	多个 Q head 共享一组 K/V
MQA	多	1	所有 Q head 共享 K/V

这会改变 K/V Shape 和 KV Cache 大小，但基本 Attention 公式不变。

本课程边界

本讲只需要认识这种变化。GQA/MQA 的工程收益、质量权衡和具体实现，等以后读相关 LLM 论文时再深入。

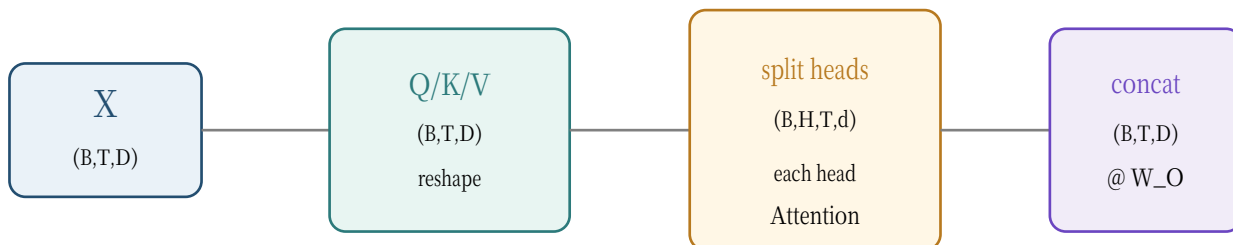
19. 与 Transformer / LLM / Agent Systems 的关系

你以后会看到	本讲对应的底层理解
num_attention_heads	H
head_dim	$d_{head} = D/H$
q_proj/k_proj/v_proj	产生总 Q/K/V
view/reshape/transpose	把总 feature 维拆成 Head 轴
attn_mask / key_padding_mask	控制 Key 可见性
out_proj / o_proj	concat 后重新混合多个 Head
GQA/MQA	减少 K/V heads

对 Agent Systems 来说，理解 Multi-Head 不代表你要研究底层模型训练，而是让你能判断长上下文、memory token、mask、KV cache、LoRA target module 和 attention analysis 实验究竟在改变哪一层机制。

20. 本讲总装与口述自检

Multi-Head Attention 数据流



Head 是拆 feature dimension，不是拆 Token；所有 Head 都看到同一序列位置。

请尝试不看答案口述：

- 1) Head 拆的是 Token 还是 Feature Dimension？
- 2) 为什么 scaling 用 $\sqrt{d_{\text{head}}}$ ？
- 3) (B, T, D) 怎样经过 reshape/transpose 变成 $(B, H, T, d_{\text{head}})$ ？
- 4) Causal Mask 与 Padding Mask 的 Shape / 语义有什么不同？
- 5) 多个 Head 输出怎样恢复成 (B, T, D) ？
- 6) W_O 的作用是什么？
- 7) D 固定时，为什么增加 Head 数通常不增加标准 QKV/O Parameter Count？
- 8) 为什么“某个 Head 看起来很像语法 Head”还不能直接当因果结论？

下一讲

第 56 讲会把 Attention 子层重新装回 Transformer Block：MLP、Residual、LayerNorm/RMSNorm、Pre-Norm/Post-Norm 将第一次和 Multi-Head Attention 汇合。